



KOCAELİ ÜNİVERSİTESİ

Mühendislik Fakültesi

Yazılım Mühendisliği Bölümü

Programlama II

Dönem Projesi Raporu

Whack-a-Mole Oyunu

AD - SOYAD : Ahmet Melih Çalış

NO : 240229027

İÇİNDEKİLER

1. Giriş.....	3
1.1. Projenin Amacı.....	3
1.2. Projenin Kapsamı.....	3
1.3. Kullanılan Teknolojiler.....	3
2. Sistem Analizi ve Gereksinimler.....	3
2.1. Fonksiyonel Gereksinimler.....	3
2.2. Veri Saklama ve Dosya Tabanlı Kalıcılık.....	4
3. Yazılım ve Fonksiyon Mimarisi.....	4
3.1. Nesne Yönelimli Programlama (OOP) Yaklaşımı.....	4
3.2. Oyun Döngüsü Mimarisi.....	4
3.3. Katmanlı Çizim Mantığı.....	5
3.4. Sınıf ve Fonksiyon Hiyerarşisi.....	5
3.4.1. Game Sınıfı Fonksiyonları.....	6
3.4.2. Mole Sınıfı Fonksiyonları.....	7
3.4.3. UIManager Sınıfı Fonksiyonları.....	7
3.5. Projede Kullanılan SDL2 Fonksiyonları Listesi.....	8
3.5.1. SDL2 Çekirdek Fonksiyonları:.....	8
3.5.2. SDL_image Fonksiyonları (Görsel İşlemleri):.....	8
3.5.3. SDL_ttf Fonksiyonları (Yazı ve Font Yönetimi):.....	8
3.5.4. SDL_mixer Fonksiyonları (Ses Motoru):.....	8
4. Algoritmalar.....	9
4.1. Çarpışma Algılama (Hit Detection) Algoritması.....	9
4.2. Dinamik Zaman Yönetimi ve Rastgelelik.....	9
4.3. Köstebek Hareket ve Sarsıntı Animasyonu.....	9
4.4. Yüksek Skor Sıralama Algoritması.....	9
5. Kullanıcı Arayüzü (UI) Geliştirmeleri.....	10
5.1. Ana Menü.....	10
5.2. Oyun İçi Ekran.....	10
5.3. Oyun Sonu Ekranı.....	11
5.4. Skor Ekranı.....	11
6. Sonuç ve Değerlendirme.....	12
7. Kaynakça.....	12

1. Giriş

1.1. Projenin Amacı

Bu proje, Programlama II dersi kapsamında geliştirilen ve oyuncu reflekslerini odağına alan, zamana karşı oynanan bir "Köstebek Vurmaca" (Whack-a-Mole) oyunudur. Projenin temel amacı, C++ dilinin nesne yönelimli programlama mimarisini kullanarak fare tıklamalarını, animasyonları ve oyun döngüsünü barındıran etkileşimli bir refleks oyunu ortaya koymaktır.

1.2. Projenin Kapsamı

Bu proje, grafiksel bir arayüz üzerinde çalışan, rastgele hedef üretimi , skor takibi ve zaman yönetimi sunan bir masaüstü oyun uygulamasını kapsamaktadır. Sistem genel hatlarıyla; oyuncu girdilerinin anlık olarak işlenmesi, algoritmalar aracılığıyla rastgele hedeflerin yönetilmesi, animasyon ve efektlerin gerçekleşmesi, çarpışma kontrollerinin yapılması ve oyun sonu verilerinin dosya sisteminde saklanması gibi temel yazılım süreçlerini sınırları içine almaktadır.

1.3. Kullanılan Teknolojiler

Proje dokümanında sunulan programlama dili ve görsel kütüphane önerileri dikkate alınarak, programlama dili için C++, grafik, yazı, görsel yükleme ve ses işlemleri için **SDL2**, **SDL2_image**, **SDL2_ttf** ve **SDL2_mixer** kütüphaneleri tercih edilmiştir.

2. Sistem Analizi ve Gereksinimler

2.1. Fonksiyonel Gereksinimler

Proje görev dokümanında belirtilen temel gereksinimler doğrultusunda sistem, oyuncuya yönelik aşağıdaki fonksiyonel gereksinimleri karşılamaktadır:

- **Oyun Süreci ve Zaman Yönetimi:**

Zamana karşı oynanan refleks tabanlı bir akış sağlanmıştır. Süre geriye doğru sayar ve süre dolduğunda oyun otomatik olarak sona erer.

- **Rastgele Delik ve Süre Yönetimi:**

Ekranda yer alan sabit deliklerden köstebekler, belirlenen algoritmalarla rastgele sıralarla ve sabit bekleme süreleriyle belirip kaybolur.

- **Delik Animasyonları:**

Köstebeklerin deliklerden çıkış ve deliklere giriş süreçleri görsel animasyonlarla (derinlik katmanları kullanılarak) desteklenmiştir.

- **Fare Tıklama Algılama ve Çarpışma Tespiti (Hit Detection):**

Kullanıcının fare tıklamaları anlık olarak algılanır. Köstebek kaybolmadan üzerine isabetli tıklanırsa matematiksel çarpışma tespit edilerek puan kazanılır, tıklanamaması veya boşa tıklanması durumunda puan kazanılmaz.

- **Görsel/İşitsel Geri Bildirim:**

Başarılı vuruşlarda aktif olan puan sayacı güncellenir, sarsıntı efekti ve vuruş sesi devreye girer.

- **Oyun Sonu ve Skor Ekranı:**

Oyun bittiğinde kullanıcıyı oyun sonu ekranı karşılar; bu ekranda oyuncunun skoru ve en yüksek skorlar listesi birlikte gösterilir.

2.2. Veri Saklama ve Dosya Tabanlı Kalıcılık

Veritabanı kullanılmayan bu projede veri kalıcılığı, highscores.txt dosyası üzerinden gerçekleştirilen dosya okuma/yazma işlemleriyle sağlanmıştır. Sistem en yüksek 5 skoru saklayacak şekilde tasarlanmıştır. highscores.txt dosyası oyunun çalıştırıldığı dizinde oluşturulmaktadır; proje CMake ile build klasörü içerisinde çalıştırıldığı için yüksek skor verileri pratikte build/highscores.txt konumunda tutulmaktadır.

3. Yazılım ve Fonksiyon Mimarisi

Proje, okunabilirliği, test edilebilirliği ve kodun tekrar kullanılabilirliğini artırmak amacıyla Nesne Yönelimli Programlama prensipleri ile tasarlanmıştır.

3.1. Nesne Yönelimli Programlama (OOP) Yaklaşımı

- **Encapsulation (Kapsülleme):**

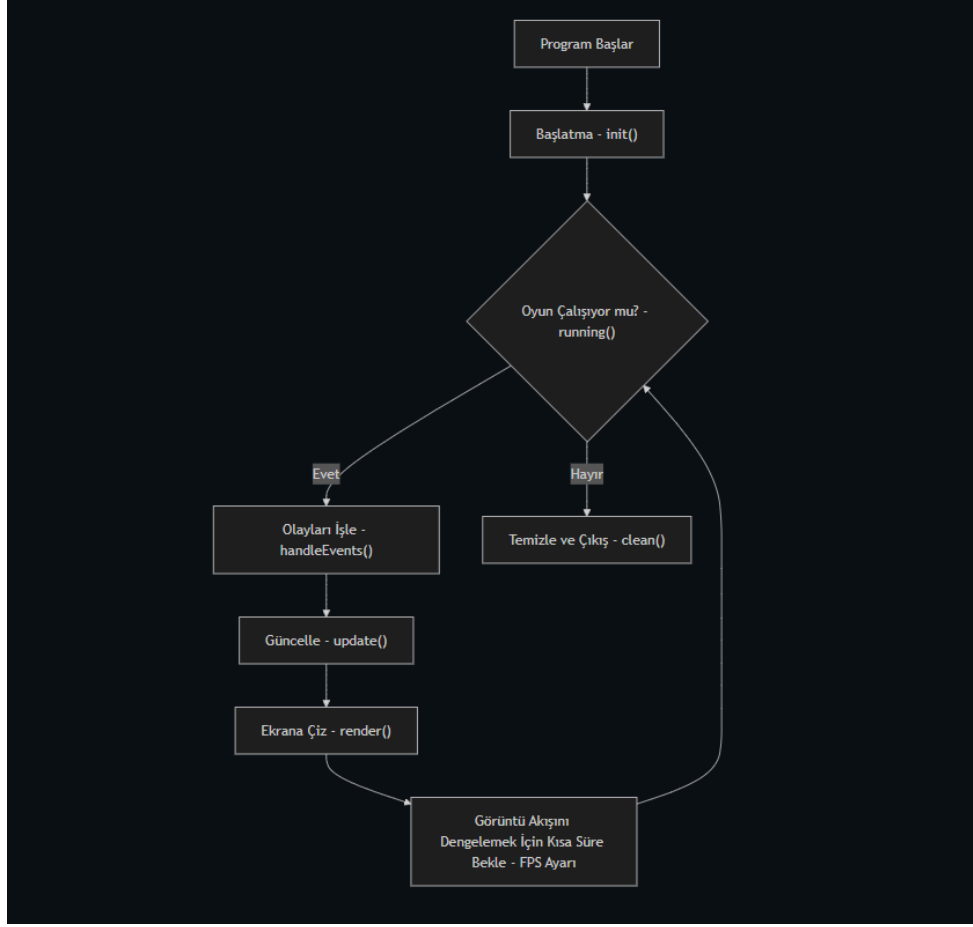
Köstebeklere ait (koordinatlar, görünürlük durumu, zamanlayıcılar) tüm değişkenler Mole sınıfı içerisinde gizlenmiş ve sadece public metotlar üzerinden erişime açılmıştır.

- **Modülerlik:**

Görsel arayüz (UI) elemanlarının çizimi UIManager sınıfına, temel oyun mantığı ise Game sınıfına devredilerek uygulama sorumlulukları ayrılmıştır.

3.2. Oyun Döngüsü Mimarisi

Sistem, saniyede 60 FPS hedefiyle çalışan sonsuz bir döngü üzerine kuruludur. Döngü içerisindeki her iterasyonda sırasıyla; kullanıcı girdileri dinlenir, oyun mantığı güncellenir ve ekrana çizim yapılır.



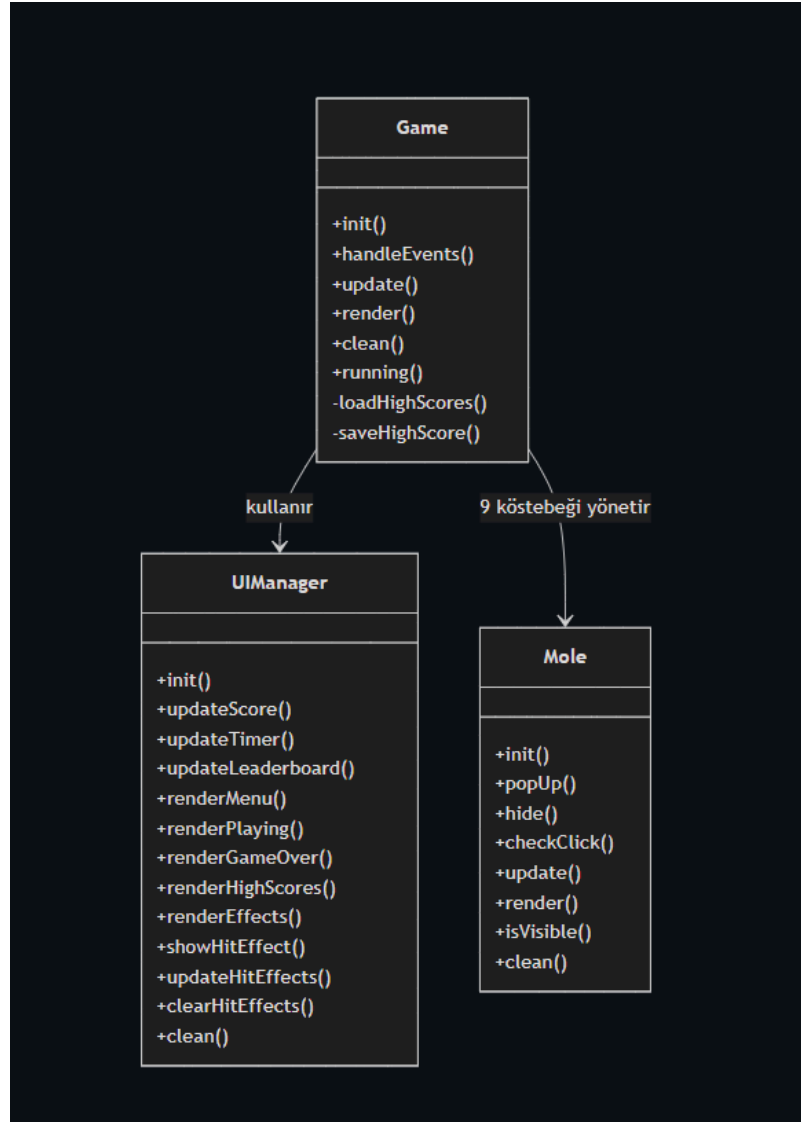
Görsel 1 : Oyun Akış Diyagramı

3.3. Katmanlı Çizim Mantığı

Grafikler ekrana yazdırılırken Z-Index (Derinlik) mantığı kullanılmıştır. Köstebek nesneleri, deliğin arka planı ile ön planı arasına çizilerek nesnelerin gerçekten deliğin içinden çıkıyormuş gibi görünmesi sağlanmıştır.

3.4. Sınıf ve Fonksiyon Hiyerarşisi

Projedeki yazılım mimarisi, üç ana sınıf üzerine inşa edilmiştir. Bu sınıfların temel fonksiyonları ve üstlendikleri sorumluluklar, aşağıda yer alan diyagram ve açıklamalarla detaylı şekilde sunulmuştur.



Görsel 2 : Sınıflar Arası İlişki

3.4.1. Game Sınıfı Fonksiyonları

Oyun döngüsünü, pencere sistemini ve temel akışı yöneten ana sınıftır.

- **init()**: SDL2 kütüphanelerini, oyun penceresini, çiziciyi (Renderer) ve ses motorunu başlatır, arka plan resimlerini yükler.
- **handleEvents()**: İşletim sisteminden gelen pencere ve fare tıklaması olaylarını dinler, buton tıklamalarını yakalayıp menü geçişlerini ve hedefe isabet durumlarını yönetir.
- **update()**: Saniye saniye akan zamanlayıcıyı, köstebeklerin çıkış zamanlamalarını, vuruş efektlerini ve animasyon durum güncellemelerini işler.
- **render()**: Arka planı ve ekranda yer alması gereken tüm katmanları çizmek üzere ilgili alt sınıflara komut gönderir.

- **clean()**: Oyun kapandığında RAM'de bellek sızıntısı olmaması için oluşturulan pencereleri, sesleri ve arka plan texture'larını yok eder.
- **running()**: Oyunun çalışmaya devam edip etmediğini kontrol eden ve main.cpp içerisindeki oyun döngüsünü ayakta tutan bool değer döndürür.
- **loadHighScores() / saveHighScore()**: Veri kalıcılığını sağlamak için highscores.txt dosyasından okuma yapar ve yeni skorları diske yazar.

3.4.2. Mole Sınıfı Fonksiyonları

Köstebek nesnelerini kapsülleyen ve durumlarını (animasyon, hitbox vb.) yöneten sınıftır.

- **init()**: Köstebeğin ve deliğin resimlerini yükleyerek ızgara (Grid) üzerindeki başlangıç koordinatlarını ayarlar.
- **popUp() / hide()**: Köstebeğin animasyon durumunu Gizli (Hidden) halden Yükselen (Rising) veya Düşen (Falling) duruma geçirir.
- **checkClick()**: Farenin X ve Y koordinatlarını alarak **SDL_PointInRect** fonksiyonu ile köstebeğin isabet alanına (Hitbox) değip değmediğini matematiksel olarak doğrular.
- **update() / render()**: Köstebeğin animasyon sürelerine göre Y ekseninde ne kadar kaydırılacağını hesaplar ve katmanlı çizim (Z-Index/Clipping) kurallarına göre ekrana basar.
- **isVisible()**: Köstebeğin o an ekranda aktif/görünür olup olmadığını (durumunun HIDDEN olup olmadığını) bildiren yardımcı fonksiyondur.
- **clean()**: Köstebek nesneleri yok edilirken bellekte kalan görsel dokuları (mole, hole, hole_front vb.) silerek RAM'i temizler.

3.4.3. UIManager Sınıfı Fonksiyonları

Ekranda görünen tüm metinleri, panelleri, butonları ve görsel efektleri yöneten sınıftır.

- **init()**: TTF font kütüphanesini başlatarak oyunda kullanılacak renkleri, başlıkları ve buton alanlarını bellekte hazırlar.
- **updateScore() / updateTimer()**: Oyuncunun anlık puanını ve geriye sayan zamanı yeni bir texture'a dönüştürerek günceller.
- **updateLeaderboard()**: Oyun bitiminde en yüksek 5 skoru sıralı liste halinde alarak ekrandaki liderlik tablosunun metin görsellerini günceller.
- **renderMenu() / renderPlaying() / renderGameOver() / renderHighScores()**: Oyunun o anki durumuna göre; ana menüyü, oyun esnasını, süre bittiğinde çıkan sonuç ekranını veya en yüksek skorlar sayfasını çizer.

- **renderEffects() / showHitEffect() / updateHitEffects() / clearHitEffects():**
Köstebek vurulduğunda ortaya çıkan "+10" puan yazılarını ve efektleri oluşturur, zamanla yukarı kaydırarak günceller ve süresi dolanları hafızadan temizler.
- **clean():** Kullanılan font nesnesini kapatır ve oluşturulan tüm buton/metin texture'larını bellekten kaldırır.
- **getPlayButtonRectangle() / getLeaderboardButtonRectangle() / getExitButtonRectangle() / getBackButtonRectangle() / getBackIconRectangle() / getGameOverRectangle():** Fare tıklamalarının butonların üzerine gelip gelmediğini kontrol etmek amacıyla Game sınıfının buton sınır koordinatlarını okumasını sağlayan metotlardır.

3.5. Projede Kullanılan SDL2 Fonksiyonları Listesi

Projenin geliştirilmesinde, grafik çizimleri, pencere yönetimi, ses çalma işlemleri ve yazı renderlama süreçleri için aşağıdaki hazır SDL2 ve alt kütüphane fonksiyonlarından da yararlanılmıştır:

3.5.1. SDL2 Çekirdek Fonksiyonları:

SDL_Init(), SDL_CreateWindow(), SDL_CreateRenderer(),
SDL_CreateTextureFromSurface(), SDL_FreeSurface(), SDL_PollEvent(),
SDL_PointInRect(), SDL_GetTicks(), SDL_GetMouseState(), SDL_SetHint(),
SDL_SetRenderDrawColor(), SDL_SetRenderDrawBlendMode(),
SDL_SetTextureColorMod(), SDL_SetTextureAlphaMod(), SDL_RenderClear(),
SDL_RenderCopy(), SDL_RenderFillRect(), SDL_RenderDrawLine(),
SDL_RenderSetClipRect(), SDL_RenderPresent(), SDL_QueryTexture(), SDL_Delay(),
SDL_DestroyTexture(), SDL_DestroyWindow(), SDL_DestroyRenderer(), SDL_Quit()

3.5.2. SDL_image Fonksiyonları (Görsel İşlemleri):

IMG_Init(), IMG_Load(), IMG_GetError()

3.5.3 SDL_ttf Fonksiyonları (Yazı ve Font Yönetimi):

TTF_Init(), TTF_OpenFont(), TTF_SetFontStyle(), TTF_RenderText_Blended(),
TTF_CloseFont(), TTF_GetError(), TTF_Quit()

3.5.4. SDL_mixer Fonksiyonları (Ses Motoru):

Mix_OpenAudio(), Mix_LoadWAV(), Mix_PlayChannel(), Mix_FreeChunk(),
Mix_GetError(), Mix_CloseAudio()

4. Algoritmalar

4.1. arpışma Algılama (Hit Detection) Algoritması

Oyuncunun hedefleri vurup vurmadığı, her fare tıklamasında çalıştırılan matematiksel bir kontrol fonksiyonu ile denetlenir. Farenin X ve Y koordinatları bir nokta (SDL_Point) olarak yakalanır ve o an aktif/yükselmiş olan köstebeğin isabet alanı (hitRectangle) içerisine düşüp düşmediği **SDL_PointInRect()** algoritması ile test edilir. Matematiksel olarak isabet başarılıysa köstebeğin isHit durumu true olarak güncellenir ve düşüş animasyonuna geçilir.

4.2. Dinamik Zaman Yönetimi ve Rastgelelik

- **Zaman Yönetimi:**

Oyun döngüsünün ve animasyonların akıcı bir şekilde çalışması için oyun hiçbir yerde durdurulmaz. Zaman hesaplamaları **SDL_GetTicks()** fonksiyonu ile milisaniye bazında farklar alınarak arka planda hesaplanır.

- **Hedef Seçimi (Rastgelelik):**

3x3'lük oyun ızgarasındaki 9 delikten hangisinden yeni bir köstebek fırlatılacağı **rand() % 9** fonksiyonu ile rastgele seçilir. Köstebeğin ekranda kalma süresi ise sabit bir süre olarak belirlenmiştir.

4.3. Köstebek Hareket ve Sarsıntı Animasyonu

Köstebeklerin delikten çıkma, inme ve vurulma anındaki hareketleri şu mantıkla hesaplanır:

- **Hareket:**

Köstebeğin yukarı çıkış ve aşağı iniş hareketleri, geçen milisaniyeye göre hesaplanır ve Y koordinatı kaydırılarak bir animasyon oluşturulur.

- **Gövde Maskeleme (Clipping):**

Köstebeğin alt gövdesinin deliğin dışına taşmaması için çizim alanı

SDL_RenderSetClipRect() ile delik sınırlarıyla kısıtlanır. Böylece nesne sadece deliğin içindeyken görünür olur.

- **Vurulma Sarsıntısı:**

Köstebek darbe aldığında, çok kısa bir süre boyunca koordinatları hızla sağa ve sola kaydırılarak görsel bir sarsıntı efekti yaratılır.

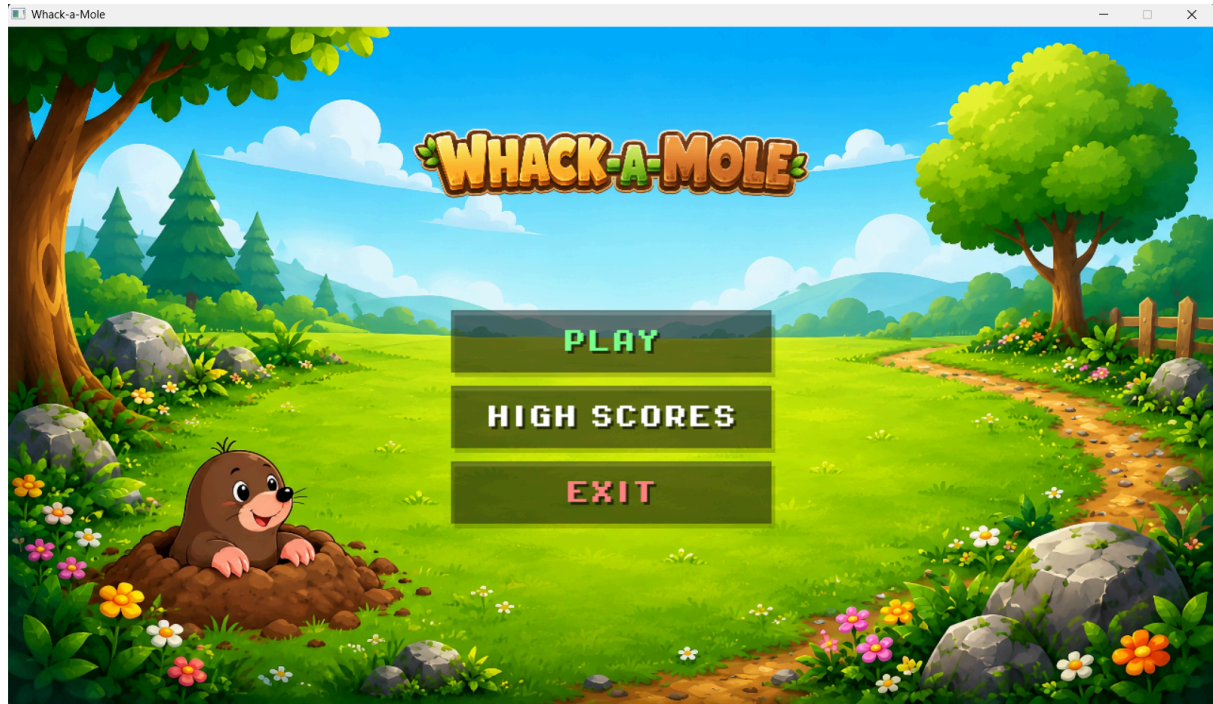
4.4. Yüksek Skor Sıralama Algoritması

Oyun bittiğinde oyuncunun elde ettiği yeni skor, en yüksek skorlar listesine eklenir. Liste, C++'ın hazır **std::sort** fonksiyonu kullanılarak büyükten küçüğe sıralanır. Sıralama

sonrasında sadece en yüksek 5 skor korunur ve bu skorlar **highscores.txt** dosyasına kaydedilerek oyun kapatılsa bile hafızada saklanması sağlanır.

5. Kullanıcı Arayüzü (UI) Geliştirmeleri

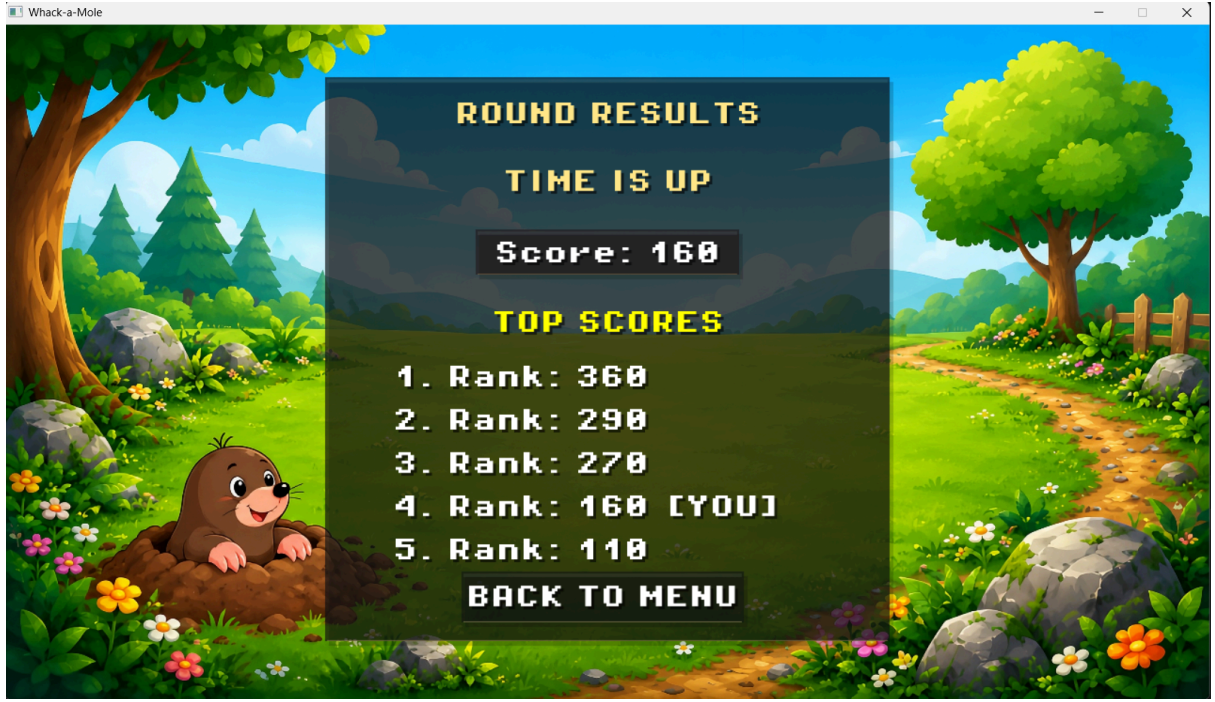
5.1. Ana Menü



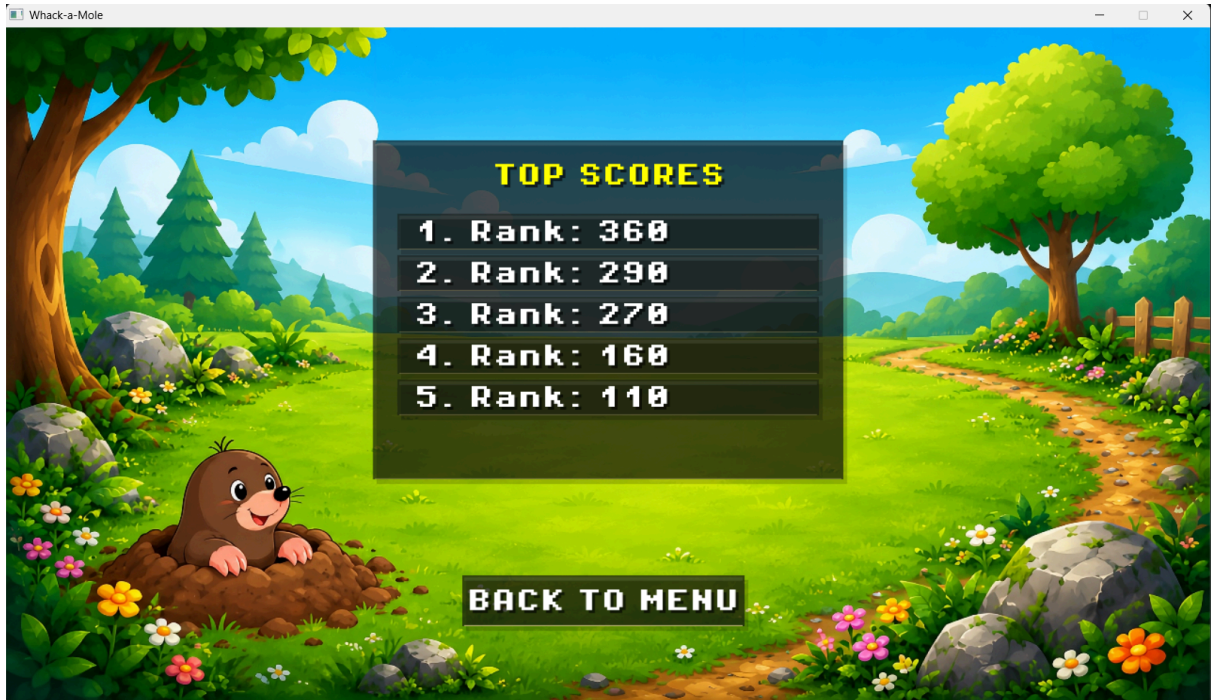
5.2. Oyun İçi Ekran



5.3. Oyun Sonu Ekranı



5.4. Skor Ekranı



6. Sonuç ve Değerlendirme

Whack-a-Mole Oyun Projesi, ödev dokümanında belirtilen fonksiyonel ve teknik gereksinimleri karşılayacak şekilde geliştirilmiştir. C++ tabanlı OOP mimarisi ile sürdürülebilir bir kod tabanı oluşturulmuş, bellek sızıntılarını önlemek adına hafıza yönetimi uygulanmıştır. Ayrıca, çarpışma algoritmaları ve SDL2 kütüphanesi sayesinde sistem, akıcı bir oyun deneyimi sunmayı hedeflemiştir. Sonuç olarak proje, oyun programlamanın temel yapı taşlarını ortaya koyan bir yazılım çözümü olmuştur.

7. Kaynakça

[1] SDL Documentation

Erişim adresi: <https://www.libsdl.org/>

[2] Lazy Foo' Productions, Beginning Game Programming v2.0.

Erişim adresi: <https://lazyfoo.net/tutorials/SDL/>

[3] C++ Reference, Standard Template Library (STL).

Erişim adresi: <https://en.cppreference.com/>

[4] Google, Gemini

Erişim adresi: <https://gemini.google.com/>

[5] OpenAI, ChatGPT

Erişim adresi: <https://chatgpt.com/>